Here's a complete response covering **both options** you requested.

Option 2: Clean Explanation for Architects (Ready to Share)

Time Series Audit (`run_timeseries_audit` **)** – Explanation for Architects

Purpose

This audit is the **time-series-specific** branch of your autonomous pre-cleaning phase. While supervised audits focus on predictive power (PPS) and clustering audits focus on variance, time series data requires special treatment because **temporal order, continuity, and sequence integrity** are the most important properties.

The goal of this audit is to:

- Detect whether the dataset respects time ordering.
- Identify columns that should be dropped (e.g., unique IDs that break grouping).
- Recommend safe, time-series-aware cleaning strategies.
- Flag downstream requirements (preserve order, stationarity testing, etc.).

This ensures the cleaning pipeline does not accidentally shuffle rows, apply mean imputation, or destroy temporal structure.

Key Design Decisions Explained

1. **Monotonicity Check** (`is_sorted = df.index.is_monotonic_increasing`)
    - Most time series models assume data is sorted by time. Unsorted data breaks lag features, rolling windows, and forecasting.
    - This is a fast first check on the current index.
2. **ID Column Detection**
    - Uses the ydata-profiling report to find unique columns (`is_unique`).
    - These are typically dropped because they add no temporal information and can interfere with grouping (e.g., per-sensor or per-product time series).

3. **Date Column Heuristic**
    - Simple keyword search for columns containing “date”, “time”, or “timestamp”.
    - This helps the pipeline know it is dealing with true temporal data.
4. **Recommended Configuration**
    - `imputation_strategy: "forward_fill"` → Standard best practice for time series. Last observation carried forward (LOCF) preserves trends and is realistic for many domains (sales, sensor data, stock prices).
    - `interpolation: "linear"` → Safe fallback for numeric gaps when forward-fill is not ideal.
    - `preserve_order: True` → Critical flag. Tells the cleaning pipeline **never** to shuffle or sort rows arbitrarily.
    - `stationarity_test_required: True` → Reminds the modeling stage to check/test for stationarity (ADF test, KPSS, etc.), which is essential for ARIMA-family models and many others.

Strengths

- Correctly shifts mindset from “correlation/variance” to “temporal integrity”.
- Recommends forward-fill, which is far better than mean/median for sequential data.
- Produces clean JSON output for easy consumption by `cleaning.py`.
- Lightweight and fast.

Limitations (Honest Assessment)

- Monotonicity check only looks at the DataFrame index (weak if time is in a column).
- Date detection is naive (keyword-based only).
- No actual gap analysis, frequency inference, or duplicate timestamp detection.
- No seasonality or trend preliminary insights.

Overall, this is a **solid foundation** for the time series path, but it should be strengthened for production-grade forecasting or anomaly detection pipelines.

Option 1: Improved & Robust Version of `run_timeseries_audit()`

Here is a **significantly better version** that keeps the spirit of your original code but adds proper temporal analysis while remaining generic and production-ready.

```python
# timeseries_audit.py  (Improved Version)
import pandas as pd
import numpy as np
import json
import os
from constants import DATASET_PATH, YDATA_REPORT_JSON,
PRE_CLEAN_AUDIT_REPORT

def run_timeseries_audit():
    print("⏳ Phase 2: Auditing for Sequential Time Series...")

    df = pd.read_csv(DATASET_PATH)

    # Load profiling report
    with open(YDATA_REPORT_JSON, 'r') as f:
        report = json.load(f)

    n_rows = len(df)

    # 1. Identify ID / useless columns
    id_cols = [col for col, s in report['variables'].items()
               if s.get('is_unique') or s.get('n_distinct') == n_rows]

    # 2. Detect potential datetime column(s)
    date_candidates = [col for col in df.columns
                       if any(k in col.lower() for k in ['date', 'time',
'timestamp', 'dt'])]

    datetime_col = None
    freq = None
    has_gaps = False
    gap_count = 0

    if date_candidates:
        for col in date_candidates:
```

```
            try:
                # Try to convert to datetime
                temp = pd.to_datetime(df[col], errors='coerce')
                if temp.notna().sum() > n_rows * 0.8:  # At least 80%
valid dates
                    datetime_col = col
                    df_sorted =
df.sort_values(by=col).reset_index(drop=True)

                    # Check monotonicity after sorting by time column
                    is_monotonic = temp.is_monotonic_increasing

                    # Infer frequency and check gaps
                    if pd.api.types.is_datetime64_any_dtype(temp):
                        freq = pd.infer_freq(temp.dropna())
                        time_diffs = temp.dropna().diff().dropna()
                        expected_step = time_diffs.median()
                        gap_count = (time_diffs > expected_step *
1.5).sum()
                        has_gaps = gap_count > 0
                    break
            except:
                continue

    # Fallback: check current index monotonicity
    is_sorted = df.index.is_monotonic_increasing if datetime_col is None
else is_monotonic

    # 3. Time-series specific recommendations
    audit_results = {
        "dataset_info": {
            "n_rows": n_rows,
            "n_columns": len(df.columns)
        },
        "config": {
            "imputation_strategy": "forward_fill",   # Best default for
most TS
            "interpolation": "linear",               # For continuous
numeric gaps
            "preserve_order": True,
            "stationarity_test_required": True,
            "resampling_required": bool(has_gaps),
            "scaling_required": False                # Many TS models
```

```
            scaling_required": False,                # Many TS models
don't need scaling
            "suggested_freq": freq
        },
        "id_cols_to_drop": id_cols,
        "temporal_consistency": {
            "has_datetime_column": bool(datetime_col),
            "datetime_column": datetime_col,
            "is_sorted": bool(is_sorted),
            "has_gaps": has_gaps,
            "gap_count": int(gap_count),
            "inferred_frequency": freq
        }
    }

    # Ensure directory exists
    os.makedirs(os.path.dirname(PRE_CLEAN_AUDIT_REPORT), exist_ok=True)

    with open(PRE_CLEAN_AUDIT_REPORT, 'w') as f:
        json.dump(audit_results, f, indent=4, default=str)  # default=str
handles pd freq

    print(f"✅ Time Series Audit complete.")
    print(f"   Datetime column detected : {datetime_col}")
    print(f"   Data is sorted           : {is_sorted}")
    print(f"   Gaps detected            : {has_gaps} ({gap_count} gaps)")
    print(f"   Inferred frequency       : {freq}")

    return audit_results


if __name__ == "__main__":
    run_timeseries_audit()
```

Key Improvements in the Enhanced Version

- Proper datetime detection and conversion.
- Actual gap analysis and frequency inference (`pd.infer_freq`).
- Better monotonicity check using the real time column.
- More informative JSON output (gap count, inferred frequency, etc.).

- Clear flags for downstream pipeline (`resampling_required` , `scaling_required: False` — because many TS models are scale-sensitive in a different way).

This version is much more useful for real time series projects (forecasting, anomaly detection, etc.).

Would you like me to also show how `cleaning.py` should behave differently based on the audit type (supervised vs clustering vs time series)? Just say the word.